МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа № 2 по курсу «Операционные системы и архитектура компьютера»

Выполнил: Щербакова М.А.
Группа: М8О-204БВ-25
Преподаватель: Н. А. Гапонов
Дата: 22.10.2026
Оценка: _____

Москва, 2026

Условие

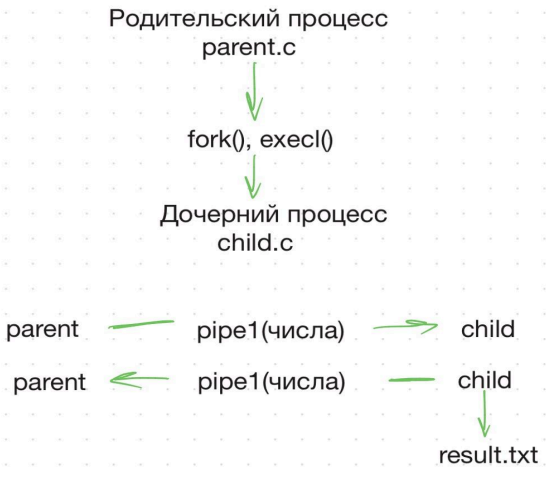
Цель работы: изучить создание дочерних процессов и организацию межпроцессного взаимодействия с помощью неименованных каналов (pipe). Получить практические навыки использования системных вызовов fork(), pipe(), dup2(), exec(), waitpid(), read() и write().

Задание: разработать на языке С программу, состоящую из родительского и дочернего процессов, реализованных в виде отдельных исполняемых файлов. Родительский процесс должен получать данные от пользователя и передавать их дочернему процессу через канал pipe1, подключённый к стандартному вводу дочернего процесса. Для передачи информации от дочернего процесса родительскому используется второй канал pipe2. Дочерний процесс должен обрабатывать полученные данные и записывать результаты в файл, имя которого пользователь вводит перед началом работы.

Вариант: пользователь вводит строки, содержащие произвольное количество целых чисел. Дочерний процесс последовательно делит первое число на все последующие и записывает результат в файл. Если среди делителей встречается ноль, дочерний процесс должен обнаружить ошибку, после чего оба процесса должны завершить работу.

Метод решения

Для выполнения задания разработаны две программы на языке С: parent.c и child.c, взаимодействующие через два неименованных канала (pipe). Родительский процесс запрашивает имя выходного файла, создаёт два канала с помощью pipe() и дочерний процесс с помощью fork(). Первый канал (pipe1) служит для передачи чисел дочернему процессу, второй (pipe2) — для получения сообщений об ошибках. В дочернем процессе функция dup2() перенаправляет стандартный ввод и поток ошибок на соответствующие каналы, после чего execl() запускает программу child. Родитель считывает строки с помощью getline() и передаёт их через pipe1. Дочерняя программа разделяет строки на числа с помощью strtok(), последовательно выполняет целочисленное деление и записывает результаты в файл. При делении на ноль дочерний процесс сообщает об ошибке через pipe2 и завершает работу, после чего завершается родительский процесс. При обычном окончании ввода родитель закрывает канал и ожидает завершения дочернего процесса с помощью waitpid().



Описание программы

Программа состоит из двух файлов на языке C: parent.c и child.c. Родительский процесс получает данные от пользователя и передаёт их дочернему через неименованный канал. Дочерний процесс выполняет целочисленное деление и записывает результаты в файл. Для передачи сообщений об ошибках используется второй канал. В данном разделе рассмотрены структура проекта, основные типы данных, функции и системные вызовы. Полный исходный код представлен в приложении А.

Структура файлов

Таблица 1 – Файлы проекта

Файл	Назначение
`parent.c`	Создание дочернего процесса, передача чисел через канал и обработка ошибок
`child.c`	Получение чисел, выполнение деления и запись результатов в файл
`Makefile`	Компиляция родительской и дочерней программ

Основные типы данных и функции

В программе используются стандартные типы данных языка C: int для хранения целых чисел и результатов вычислений, char для работы со строками, pid_t для идентификатора дочернего процесса, size_t для размеров буферов и ssize_t для результатов операций чтения и записи. Для обработки входных данных используются функции getline(), strlen(), strtok() и atoi(). Функция getline() считывает строки произвольной длины, strtok() разделяет их на отдельные числа, а atoi() преобразует строки в целые числа. Для работы с выходным файлом используются функции fopen(), fprintf() и fclose(). Межпроцессное взаимодействие реализовано с помощью системных вызовов, описанных в следующем подразделе.

Используемые системные вызовы

Таблица 2 – Используемые системные вызовы

Системный вызов	Назначение в программе
`int pipe(int fd[2])`	Создание двух каналов для обмена данными между родительским и дочерним процессами.
`pid_t fork(void)`	Создание дочернего процесса.
`int dup2(int old, int new)`	Перенаправление стандартного ввода и потока ошибок дочернего процесса на соответствующие каналы.
`int execl(const char *path, const char *arg, ...)`	Запуск программы `child` в дочернем процессе.

Системный вызов	Назначение в программе
`ssize_t read(int fd, void *buf, size_t count)`	Чтение сообщений об ошибках из второго канала.
`ssize_t write(int fd, const void *buf, size_t count)`	Передача введённых чисел дочернему процессу через первый канал.
`int close(int fd)`	Закрытие неиспользуемых концов каналов.
`pid_t waitpid(pid_t pid, int *status, int options)`	Ожидание завершения дочернего процесса и получение его статуса.
`int select(int nfds, fd_set *readfds, fd_set *writefds, fd_set *exceptfds, struct timeval *timeout)`	Ожидание появления данных от пользователя или сообщения от дочернего процесса.

Ключевые фрагменты кода

Для обмена данными между процессами создаются два неименованных канала. Затем с помощью fork() создаётся дочерний процесс.

Листинг 1 – Создание канала и дочернего процесса (parent.c)

```
int pipe1[2], pipe2[2];
pipe(pipe1);
pipe(pipe2);
pid_t pid = fork();
```

В дочернем процессе стандартный ввод и поток ошибок перенаправляются на соответствующие каналы. После этого запускается программа child.

Листинг 2 – Перенаправление потоков (`parent.c`)

```
dup2(pipe1[0], STDIN_FILENO);
dup2(pipe2[1], STDERR_FILENO);
execl("./child", "child", filename, NULL);
```

Дочерняя программа разделяет полученную строку на числа и последовательно выполняет целочисленное деление.

Листинг 3 – Выполнение вычислений (child.c)

```
char *num = strtok(line, " \t\n");
int result = atoi(num);
```

```
while ((num = strtok(NULL, " \t\n")) != NULL) {
    int x = atoi(num);
    if (x == 0) {
        fprintf(stderr, "Division by zero\n");
        free(line);
        fclose(file);
        return 2;
    }
    result = result / x;
}
fprintf(file, "%d\n", result);
```

После окончания передачи данных родительский процесс ожидает завершения дочернего процесса и проверяет его статус.

Листинг 4 – Ожидание завершения дочернего процесса (parent.c)

```
int status;
if (waitpid(pid, &status, 0) == -1) {
    perror("waitpid");
    return 1;
}
if (!WIFEXITED(status) || WEXITSTATUS(status) != 0) {
    error = 1;
}
```

Результаты

В ходе выполнения лабораторной работы были скомпилированы и протестированы программы parent.c и child.c. Проверка проводилась на нескольких наборах целых чисел. В первом тесте проверялась правильность последовательного целочисленного деления и записи результатов в файл. Во втором тесте проверялась обработка ошибки деления на ноль.

Тест 1. Корректные входные данные

При запуске программы пользователь указал выходной файл result.txt и ввёл следующие строки:

```
100 2 5
81 3 3
120 2 3 4
50 2
```

После завершения ввода программа записала в файл result.txt следующие результаты:

```
10
9
5
25
```

Все результаты соответствуют последовательному целочисленному делению.

Тест 2. Деление на ноль

Для проверки обработки ошибок был указан файл zero.txt и введена строка:

```
100 2 0
```

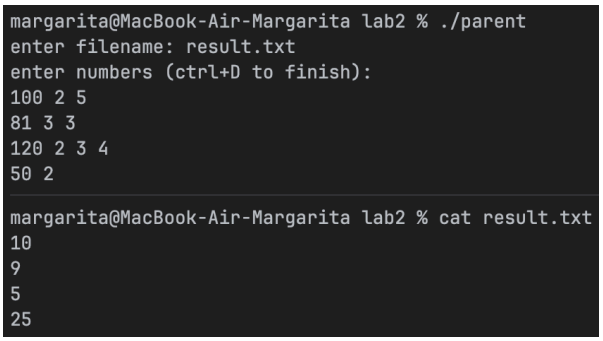
Программа обнаружила деление на ноль и вывела сообщение:

```
Division by zero
```

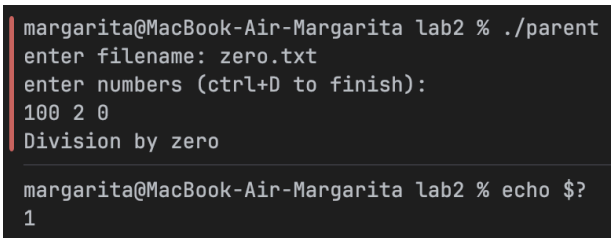
После обнаружения ошибки оба процесса завершили выполнение. Код завершения родительской программы составил 1.

Таким образом, тестирование подтвердило правильность вычислений, запись результатов в файл и обработку деления на ноль.

Скриншот 1 – правильная работа



Скриншот 2 – деление на ноль



Демонстрация работы strace

Для анализа системных вызовов программа была запущена в среде Ubuntu 24.04 с помощью утилиты strace. Использовалась команда: `strace -f -o strace.txt ./parent`.

Параметр `-f` позволяет отслеживать системные вызовы дочернего процесса, а параметр `-o` сохраняет трассировку в файл. Полный вывод приведён в приложении Б.

1. Создание неименованных каналов

```
1099  pipe2([3, 4], 0) = 0
1099  pipe2([5, 6], 0) = 0
```

Родительский процесс создаёт два неименованных канала. Первый канал используется для передачи данных дочернему процессу, второй — для передачи сообщений об ошибках от дочернего процесса родительскому. Дескрипторы 3 и 5 предназначены для

чтения, 4 и 6 — для записи. Возвращаемое значение 0 означает успешное создание каналов.

2. Создание дочернего процесса

```
1099  clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD,
child_tidptr=0xffffb30a7fb0) = 1100
```

Вызов `fork()` реализуется библиотекой glibc через системный вызов `clone()`. В результате создаётся дочерний процесс с PID 1100. PID родительского процесса — 1099.

3. Запуск дочерней программы

```
1100  execve("./child", ["child", "result_linux.txt"], 0xffffd9153c98 /* 8 vars */) = 0
```

Дочерний процесс запускает программу `child`, передавая ей имя выходного файла `result_linux.txt` в качестве аргумента командной строки.

4. Передача данных через канал

```
1099  write(4, "100 2 5\n", 8) = 8
1099  write(4, "81 3 3\n", 7) = 7
1099  write(4, "120 2 3 4\n", 10) = 10
1099  write(4, "50 2\n", 5) = 5
```

Родительский процесс записывает введённые строки в первый канал с помощью системного вызова `write()`. Дескриптор 4 соответствует концу канала для записи. Возвращаемые значения показывают количество успешно записанных байтов.

5. Запись результатов вычислений

```
1100  write(3, "10\n9\n5\n25\n", 10) = 10
```

Дочерний процесс записывает результаты целочисленного деления в выходной файл. Для введённых строк получены значения 10, 9, 5 и 25 соответственно.

6. Завершение дочернего процесса

```
1099  wait4(1100, [{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 1100
```

Родительский процесс ожидает завершения дочернего процесса. Условие `WIFEXITED(s)` подтверждает нормальное завершение, а `WEXITSTATUS(s) == 0` означает успешное выполнение программы.

Таким образом, трассировка подтверждает создание двух неименованных каналов, запуск дочернего процесса, передачу данных между процессами и корректное завершение программы.

Выводы

В ходе лабораторной работы были изучены основные способы создания процессов и организации межпроцессного взаимодействия в операционной системе. На языке C

разработаны две программы, взаимодействующие с помощью неименованных каналов pipe. Для создания дочернего процесса использовался fork(), а для запуска отдельной программы — execl(). В результате тестирования была подтверждена правильность выполнения целочисленного деления и записи результатов в файл. Также была проверена обработка ошибки деления на ноль, при которой оба процесса завершают работу. Таким образом, цель лабораторной работы была достигнута.

Приложение А. Исходный код программы

Листинг А.1 – Файл parent.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>
#include <errno.h>
#include <sys/select.h>
#include <signal.h>

int main(void) {
    char filename[256];
    printf("enter filename: ");
    fflush(stdout);
    if (fgets(filename, sizeof(filename), stdin) == NULL) {
        return 1;
    }
    filename[strcspn(filename, "\n")] = '\0';
    if (filename[0] == '\0') {
        printf("empty filename\n");
        return 1;
    }
    int pipe1[2], pipe2[2];
    if (pipe(pipe1) == -1) {
        perror("pipe1");
        return 1;
    }
    if (pipe(pipe2) == -1) {
        perror("pipe2");
        close(pipe1[0]);
        close(pipe1[1]);
        return 1;
    }
    signal(SIGPIPE, SIG_IGN);
    pid_t pid = fork();
    if (pid == -1) {
        perror("fork");
        close(pipe1[0]);
        close(pipe1[1]);
        close(pipe2[0]);
        close(pipe2[1]);
        return 1;
    }
    if (pid == 0) {
        close(pipe1[1]);
        close(pipe2[0]);
        if (dup2(pipe1[0], STDIN_FILENO) == -1) {
            perror("dup2");
            _exit(1);
        }
        if (dup2(pipe2[1], STDERR_FILENO) == -1) {
            perror("dup2");
            _exit(1);
        }
        close(pipe1[0]);
        close(pipe2[1]);
        execl("./child", "child", filename, NULL);
        perror("execl");
```

```
        _exit(1);
    }
    close(pipe1[0]);
    close(pipe2[1]);
    printf("enter numbers (ctrl+D to finish):\n");
    setvbuf(stdin, NULL, _IONBF, 0);
    char *line = NULL;
    size_t size = 0;
    int running = 1;
    int error = 0;
    while (running) {
        fd_set fds;
        FD_ZERO(&fds);
        FD_SET(STDIN_FILENO, &fds);
        FD_SET(pipe2[0], &fds);
        int maxfd = pipe2[0] > STDIN_FILENO ? pipe2[0] : STDIN_FILENO;
        if (select(maxfd + 1, &fds, NULL, NULL, NULL) == -1) {
            if (errno == EINTR) {
                continue;
            }
            perror("select");
            error = 1;
            break;
        }
        if (FD_ISSET(pipe2[0], &fds)) {
            char buffer[256];
            ssize_t n = read(pipe2[0], buffer, sizeof(buffer) - 1);
            if (n > 0) {
                buffer[n] = '\0';
                fprintf(stderr, "%s", buffer);
                error = 1;
                running = 0;
            } else if (n == 0) {
                running = 0;
            } else if (errno != EINTR) {
                perror("read");
                error = 1;
                running = 0;
            }
        }
        if (running && FD_ISSET(STDIN_FILENO, &fds)) {
            if (getline(&line, &size, stdin) == -1) {
                break;
            }
            size_t len = strlen(line);
            size_t sent = 0;
            while (sent < len) {
                ssize_t n = write(pipe1[1], line + sent, len - sent);
                if (n == -1) {
                    if (errno == EINTR) {
                        continue;
                    }
                    if (errno != EPIPE) {
                        perror("write");
                    }
                    error = 1;
                    running = 0;
                    break;
                }
                sent += n;
```

```
            }
        }
    }
    free(line);
    close(pipe1[1]);
    char buffer[256];
    ssize_t n;
    while ((n = read(pipe2[0], buffer, sizeof(buffer))) > 0) {
        fwrite(buffer, 1, n, stderr);
        error = 1;
    }
    if (n == -1) {
        perror("read");
        error = 1;
    }
    close(pipe2[0]);
    int status;
    if (waitpid(pid, &status, 0) == -1) {
        perror("waitpid");
        return 1;
    }
    if (!WIFEXITED(status) || WEXITSTATUS(status) != 0) {
        error = 1;
    }
    return error;
}
```

Листинг А.2 – Файл child.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(int argc, char *argv[]) {
    if (argc < 2) {
        return 1;
    }
    FILE *file = fopen(argv[1], "w");
    if (file == NULL) {
        perror("File error");
        return 1;
    }
    char *line = NULL;
    size_t size = 0;
    while (getline(&line, &size, stdin) != -1) {
        char *num = strtok(line, " \t\n");
        if (num == NULL) {
            continue;
        }
        int result = atoi(num);
        while ((num = strtok(NULL, " \t\n")) != NULL) {
            int x = atoi(num);
            if (x == 0) {
                fprintf(stderr, "Division by zero\n");
                free(line);
                fclose(file);
                return 2;
            }
            result = result / x;
        }
```

```
        fprintf(file, "%d\n", result);
    }
    free(line);
    fclose(file);
    return 0;
}
```

Листинг А.3 – Файл Makefile

```
CC = clang
CFLAGS = -Wall -Wextra

all: parent child
parent: parent.c
        $(CC) $(CFLAGS) parent.c -o parent

child: child.c
        $(CC) $(CFLAGS) child.c -o child

clean:
        rm -f parent child
```

Приложение Б. Полный вывод strace

Листинг Б.1 – Вывод strace -f ./parent

```
1099  execve("./parent", ["./parent"], 0xfffff6e437e8 /* 8 vars */) = 0
1099  brk(NULL)                         = 0xaaaaee07e000
1099  mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0xffffb30a7000
1099  faccessat(AT_FDCWD, "/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or
directory)
1099  openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
1099  fstat(3, {st_mode=S_IFREG|0644, st_size=8407, ...}) = 0
1099  mmap(NULL, 8407, PROT_READ, MAP_PRIVATE, 3, 0) = 0xffffb30a4000
1099  close(3)                          = 0
1099  openat(AT_FDCWD, "/lib/aarch64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
1099  read(3,
"\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0\267\0\1\0\0\0\360\206\2\0\0\0\0\0"..., 832) = 832
1099  fstat(3, {st_mode=S_IFREG|0755, st_size=1722920, ...}) = 0
1099  mmap(NULL, 1892240, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_DENYWRITE, -1, 0) =
0xffffb2ea0000
1099  mmap(0xffffb2ea0000, 1826704, PROT_READ|PROT_EXEC,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0) = 0xffffb2ea0000
1099  munmap(0xffffb305e000, 65424)     = 0
1099  mprotect(0xffffb303a000, 77824, PROT_NONE) = 0
1099  mmap(0xffffb304d000, 20480, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x19d000) = 0xffffb304d000
1099  mmap(0xffffb3052000, 49040, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0xffffb3052000
1099  close(3)                          = 0
1099  set_tid_address(0xffffb30a7fb0)   = 1099
1099  set_robust_list(0xffffb30a7fc0, 24) = 0
1099  rseq(0xffffb30a8600, 0x20, 0, 0xd428bc00) = 0
1099  mprotect(0xffffb304d000, 12288, PROT_READ) = 0
1099  mprotect(0xaaaabbf2f000, 4096, PROT_READ) = 0
1099  mprotect(0xffffb30ac000, 8192, PROT_READ) = 0
1099  prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) =
0
1099  munmap(0xffffb30a4000, 8407)      = 0
1099  fstat(1, {st_mode=S_IFCHR|0620, st_rdev=makedev(0x88, 0), ...}) = 0
1099  getrandom("\xab\x93\x8a\x9e\xf9\x18\x5d\x86", 8, GRND_NONBLOCK) = 8
1099  brk(NULL)                         = 0xaaaaee07e000
1099  brk(0xaaaaee09f000)               = 0xaaaaee09f000
1099  write(1, "enter filename: ", 16)  = 16
1099  fstat(0, {st_mode=S_IFCHR|0620, st_rdev=makedev(0x88, 0), ...}) = 0
1099  read(0, "result_linux.txt\n", 1024) = 17
1099  pipe2([3, 4], 0)                  = 0
1099  pipe2([5, 6], 0)                  = 0
1099  rt_sigaction(SIGPIPE, {sa_handler=SIG_IGN, sa_mask=[PIPE], sa_flags=SA_RESTART},
{sa_handler=SIG_DFL, sa_mask=[], sa_flags=0}, 8) = 0
1099  clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD,
child_tidptr=0xffffb30a7fb0) = 1100
1099  close(3 <unfinished ...>
1100  set_robust_list(0xffffb30a7fc0, 24 <unfinished ...>
1099  <... close resumed>)              = 0
1100  <... set_robust_list resumed>)    = 0
1099  close(6 <unfinished ...>
1100  close(4 <unfinished ...>
1099  <... close resumed>)              = 0
1100  <... close resumed>)              = 0
1099  write(1, "enter numbers (ctrl+D to finish)"..., 34 <unfinished ...>
1100  close(5 <unfinished ...>
```

```
1099  <... write resumed>)              = 34
1100  <... close resumed>)              = 0
1099  pselect6(6, [0 5], NULL, NULL, NULL, NULL <unfinished ...>
1100  dup3(3, 0, 0)                     = 0
1100  dup3(6, 2, 0)                     = 2
1100  close(3)                          = 0
1100  close(6)                          = 0
1100  execve("./child", ["child", "result_linux.txt"], 0xffffd9153c98 /* 8 vars */) = 0
1100  brk(NULL)                         = 0xaaaae715a000
1100  mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0xffffa2cec000
1100  faccessat(AT_FDCWD, "/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or
directory)
1100  openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
1100  fstat(3, {st_mode=S_IFREG|0644, st_size=8407, ...}) = 0
1100  mmap(NULL, 8407, PROT_READ, MAP_PRIVATE, 3, 0) = 0xffffa2ce9000
1100  close(3)                          = 0
1100  openat(AT_FDCWD, "/lib/aarch64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
1100  read(3,
"\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0\267\0\1\0\0\0\360\206\2\0\0\0\0\0"..., 832) = 832
1100  fstat(3, {st_mode=S_IFREG|0755, st_size=1722920, ...}) = 0
1100  mmap(NULL, 1892240, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_DENYWRITE, -1, 0) =
0xffffa2ae5000
1100  mmap(0xffffa2af0000, 1826704, PROT_READ|PROT_EXEC,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0) = 0xffffa2af0000
1100  munmap(0xffffa2ae5000, 45056)     = 0
1100  munmap(0xffffa2cae000, 20368)     = 0
1100  mprotect(0xffffa2c8a000, 77824, PROT_NONE) = 0
1100  mmap(0xffffa2c9d000, 20480, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x19d000) = 0xffffa2c9d000
1100  mmap(0xffffa2ca2000, 49040, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0xffffa2ca2000
1100  close(3)                          = 0
1100  set_tid_address(0xffffa2cecfb0)   = 1100
1100  set_robust_list(0xffffa2cecfc0, 24) = 0
1100  rseq(0xffffa2ced600, 0x20, 0, 0xd428bc00) = 0
1100  mprotect(0xffffa2c9d000, 12288, PROT_READ) = 0
1100  mprotect(0xaaaaada4f000, 4096, PROT_READ) = 0
1100  mprotect(0xffffa2cf1000, 8192, PROT_READ) = 0
1100  prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) =
0
1100  munmap(0xffffa2ce9000, 8407)      = 0
1100  getrandom("\x7f\xbd\x4f\x5e\xe5\xb5\x4e\x64", 8, GRND_NONBLOCK) = 8
1100  brk(NULL)                         = 0xaaaae715a000
1100  brk(0xaaaae717b000)               = 0xaaaae717b000
1100  openat(AT_FDCWD, "result_linux.txt", O_WRONLY|O_CREAT|O_TRUNC, 0666) = 3
1100  fstat(0, {st_mode=S_IFIFO|0600, st_size=0, ...}) = 0
1100  read(0,  <unfinished ...>
1099  <... pselect6 resumed>)           = 1 (in [0])
1099  read(0, "1", 1)                   = 1
1099  read(0, "0", 1)                   = 1
1099  read(0, "0", 1)                   = 1
1099  read(0, " ", 1)                   = 1
1099  read(0, "2", 1)                   = 1
1099  read(0, " ", 1)                   = 1
1099  read(0, "5", 1)                   = 1
1099  read(0, "\n", 1)                  = 1
1099  write(4, "100 2 5\n", 8)          = 8
1100  <... read resumed>"100 2 5\n", 4096) = 8
1099  pselect6(6, [0 5], NULL, NULL, NULL, NULL <unfinished ...>
```

```
1100  fstat(3, {st_mode=S_IFREG|0644, st_size=0, ...}) = 0
1100  read(0,  <unfinished ...>
1099  <... pselect6 resumed>)           = 1 (in [0])
1099  read(0, "8", 1)                   = 1
1099  read(0, "1", 1)                   = 1
1099  read(0, " ", 1)                   = 1
1099  read(0, "3", 1)                   = 1
1099  read(0, " ", 1)                   = 1
1099  read(0, "3", 1)                   = 1
1099  read(0, "\n", 1)                  = 1
1099  write(4, "81 3 3\n", 7)           = 7
1100  <... read resumed>"81 3 3\n", 4096) = 7
1099  pselect6(6, [0 5], NULL, NULL, NULL, NULL <unfinished ...>
1100  read(0,  <unfinished ...>
1099  <... pselect6 resumed>)           = 1 (in [0])
1099  read(0, "1", 1)                   = 1
1099  read(0, "2", 1)                   = 1
1099  read(0, "0", 1)                   = 1
1099  read(0, " ", 1)                   = 1
1099  read(0, "2", 1)                   = 1
1099  read(0, " ", 1)                   = 1
1099  read(0, "3", 1)                   = 1
1099  read(0, " ", 1)                   = 1
1099  read(0, "4", 1)                   = 1
1099  read(0, "\n", 1)                  = 1
1099  write(4, "120 2 3 4\n", 10)       = 10
1100  <... read resumed>"120 2 3 4\n", 4096) = 10
1099  pselect6(6, [0 5], NULL, NULL, NULL, NULL <unfinished ...>
1100  read(0,  <unfinished ...>
1099  <... pselect6 resumed>)           = 1 (in [0])
1099  read(0, "5", 1)                   = 1
1099  read(0, "0", 1)                   = 1
1099  read(0, " ", 1)                   = 1
1099  read(0, "2", 1)                   = 1
1099  read(0, "\n", 1)                  = 1
1099  write(4, "50 2\n", 5)             = 5
1100  <... read resumed>"50 2\n", 4096) = 5
1099  pselect6(6, [0 5], NULL, NULL, NULL, NULL <unfinished ...>
1100  read(0,  <unfinished ...>
1099  <... pselect6 resumed>)           = 1 (in [0])
1099  read(0, "", 1)                    = 0
1099  close(4)                          = 0
1100  <... read resumed>"", 4096)       = 0
1099  read(5,  <unfinished ...>
1100  write(3, "10\n9\n5\n25\n", 10)    = 10
1100  close(3)                          = 0
1100  exit_group(0)                     = ?
1099  <... read resumed>"", 256)        = 0
1099  close(5 <unfinished ...>
1100  +++ exited with 0 +++
1099  <... close resumed>)              = 0
1099  --- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=1100, si_uid=0,
si_status=0, si_utime=0, si_stime=0} ---
1099  wait4(1100, [{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 1100
1099  exit_group(0)                     = ?
1099  +++ exited with 0 +++
```